SQL Level 6: JOINS

Topic-wise notes and practice question solutions

Sample Tables Used in All Examples

A small company and shop database. NULL means 'no value'. Notice the deliberate gaps: employees with no department, a department with no employees, customers who never ordered, and a product that was never sold.

departments

dept_id	dept_name
1	IT
2	HR
3	Sales
4	Finance

employees

emp_id	emp_name	dept_id	manager_id	salary
1	Farah	1	NULL	120,000
2	Arun	1	1	85,000
3	Bhavana	1	1	90,000
4	Indira	2	1	60,000
5	Gita	2	4	30,000
6	Meera	3	1	70,000
7	Jai	3	6	30,000
8	Ravi	NULL	1	45,000
9	Sneha	NULL	6	40,000

courses

course_id	course_name
1	SQL
2	Python
3	Java

students

student_id	student_name	course_id
1	Anil	1
2	Bindu	1
3	Chitra	2
4	Deepak	2
5	Esha	NULL

regions

region_id	region_name
1	South
2	North
3	East
4	West

customers

customer_id	customer_name	region_id
1	Ramesh	1
2	Suresh	2
3	Lakshmi	1
4	Kiran	3
5	Priya	2
6	Vikram	4

categories

category_id	category_name
1	Electronics
2	Accessories
3	Furniture

products

product_id	product_name	category_id	price
1	Laptop	1	60,000
2	Phone	1	20,000
3	Mouse	2	500
4	Keyboard	2	1,500
5	Chair	3	5,000
6	Desk	3	8,000

orders

order_id	customer_id	order_date
1	1	2026-01-05
2	1	2026-02-10
3	2	2026-02-12
4	3	2026-03-01
5	4	2026-03-15
6	2	2026-03-20

order_items

item_id	order_id	product_id	quantity
1	1	1	1
2	1	3	2
3	2	2	1
4	3	4	1
5	3	3	1
6	4	5	2
7	5	1	1
8	5	4	2
9	6	2	2

PART A: TOPICS

Topic 1: What is JOIN?

What it is: Data is split across several tables to avoid repeating it (employee rows hold only a dept_id, while department names live in the departments table). A JOIN combines rows from two or more tables into one result, matching them on a related column.

Key ideas

- **Primary key**: uniquely identifies a row (departments.dept_id).
- **Foreign key**: a column that points to another table's primary key (employees.dept_id).
- **ON clause**: the matching rule, for example `e.dept_id = d.dept_id`.
- **Table alias**: a short name such as `employees e`, so you can write e.emp_name instead of employees.emp_name.
- **Qualify columns** with the alias (e.dept_id) when the same column name exists in both tables.

Syntax

```
SELECT   a.column, b.column
FROM     table_a a
JOIN_TYPE table_b b
         ON a.key = b.key;
```

Summary of join types

Join	Rows returned
INNER JOIN	Only rows that match in both tables
LEFT JOIN	All rows from the left table, plus matches from the right (NULL where no match)
RIGHT JOIN	All rows from the right table, plus matches from the left (NULL where no match)
CROSS JOIN	Every row of one table paired with every row of the other (no ON clause)
SELF JOIN	A table joined to itself, using two different aliases

Example: Without a join vs with a join

SQL Query

```
SELECT e.emp_name, e.dept_id, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;
```

Output (7 rows)

emp_name	dept_id	dept_name
Farah	1	IT
Arun	1	IT
Bhavana	1	IT
Indira	2	HR
Gita	2	HR
Meera	3	Sales
Jai	3	Sales

Explanation

Without a join you only see dept_id numbers (1, 2, 3). The join looks up each dept_id in departments and brings back the readable department name. Ravi and Sneha are missing because their dept_id is NULL and matches nothing.

Topic 2: INNER JOIN

What it is: Returns only the rows where the ON condition matches in **both** tables. Rows without a match on either side are dropped. JOIN on its own means INNER JOIN.

```
SELECT columns
FROM table_a a
INNER JOIN table_b b ON a.key = b.key;
```

Example 1: Employees with department names

SQL Query

```
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

Output (7 rows)

emp_name	dept_name
Farah	IT
Arun	IT
Bhavana	IT
Indira	HR
Gita	HR
Meera	Sales
Jai	Sales

Explanation

9 employees exist, but only 7 have a matching department. Ravi and Sneha (no dept_id) are excluded, and the Finance department also disappears because no employee points to it.

Example 2: Students with course names

SQL Query

```
SELECT s.student_name, c.course_name
FROM students s
INNER JOIN courses c ON s.course_id = c.course_id;
```

Output (4 rows)

student_name	course_name
Anil	SQL
Bindu	SQL
Chitra	Python
Deepak	Python

Explanation

Esha has no course, and the Java course has no students, so neither appears. Only matching pairs are returned.

Topic 3: LEFT JOIN

What it is: Returns **all** rows from the left table (the one after FROM). If a left row has no match in the right table, the right-side columns are filled with NULL. Also called LEFT OUTER JOIN.

```
SELECT columns
FROM table_a a
LEFT JOIN table_b b ON a.key = b.key;
```

Example 1: All employees, with department if any

SQL Query

```
SELECT e.emp_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

Output (9 rows)

emp_name	dept_name
Farah	IT
Arun	IT
Bhavana	IT
Indira	HR
Gita	HR
Meera	Sales
Jai	Sales
Ravi	NULL
Sneha	NULL

Explanation

Every employee is kept. Ravi and Sneha have no department, so dept_name shows NULL for them.

Example 2: Find unmatched rows (anti-join)

SQL Query

```
SELECT e.emp_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

Output (2 rows)

emp_name
Ravi
Sneha

Explanation

After a LEFT JOIN, rows with no match have NULL in the right table's key. Filtering WHERE right_key IS NULL keeps only the unmatched left rows. This 'LEFT JOIN + IS NULL' pattern is how you find 'never', 'without' or 'missing' records.

Common mistake

```
SELECT c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= '2026-03-01';
```

A WHERE condition on the right table removes the NULL rows, silently turning the LEFT JOIN into an INNER JOIN. Customers with no orders vanish.

Correct version

```
SELECT c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
 AND o.order_date >= '2026-03-01';
```

Putting the condition inside ON filters the right table first, so all customers are still kept.

Topic 4: RIGHT JOIN

What it is: The mirror image of LEFT JOIN. Returns **all** rows from the right table, with NULL on the left side where there is no match. A RIGHT JOIN can always be rewritten as a LEFT JOIN by swapping the table order, which is why many people prefer LEFT JOIN only.

Note: RIGHT JOIN works in MySQL, PostgreSQL, SQL Server and Oracle. SQLite supports it from version 3.39.

Example 1: All departments, with their employees

SQL Query

```
SELECT d.dept_name, e.emp_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

Output (8 rows)

dept_name	emp_name
IT	Farah
IT	Arun
IT	Bhavana
HR	Indira
HR	Gita
Sales	Meera
Sales	Jai
Finance	NULL

Explanation

Every department is kept, including Finance, which has no employees, so emp_name is NULL for it.

Example 2: Same result using LEFT JOIN (tables swapped)

SQL Query

```
SELECT d.dept_name, e.emp_name
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id;
```

Output (8 rows)

dept_name	emp_name
IT	Arun
IT	Bhavana
IT	Farah
HR	Gita
HR	Indira
Sales	Jai
Sales	Meera
Finance	NULL

Explanation

Putting departments first and using LEFT JOIN returns the same rows as Example 1. Only the order of the rows may differ.

Topic 5: CROSS JOIN

What it is: Pairs every row of the first table with every row of the second (a Cartesian product). There is no ON clause. If table A has m rows and table B has n rows, the result has m x n rows.

```
SELECT columns
FROM table_a
CROSS JOIN table_b;
```

Example 1: Every region with every product category

SQL Query

```
SELECT r.region_name, c.category_name
FROM regions r
CROSS JOIN categories c;
```

Output (12 rows)

region_name	category_name
South	Electronics
South	Accessories
South	Furniture
North	Electronics
North	Accessories
North	Furniture
East	Electronics
East	Accessories
East	Furniture
West	Electronics
West	Accessories
West	Furniture

Explanation

4 regions x 3 categories = 12 combinations. Useful for building a full planning grid, for instance to later check which region-category pairs have no sales.

Example 2: Counting the size of a cross join

SQL Query

```
SELECT COUNT(*) AS total_rows
FROM employees
CROSS JOIN departments;
```

Output (1 row)

total_rows
36

Explanation

9 employees x 4 departments = 36 rows. Be careful with large tables: 10,000 x 10,000 rows would produce 100 million rows.

Warning: forgetting the ON condition in an ordinary join (old style `FROM a, b` without WHERE) accidentally gives a cross join.

Topic 6: SELF JOIN

What it is: A table joined to itself. It is used when rows in a table relate to other rows in the same table, such as employee and manager (both are employees). Because the same table appears twice, you **must** give it two different aliases.

Example 1: Employee and manager names

SQL Query

```
SELECT e.emp_name AS employee, m.emp_name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

Output (9 rows)

employee	manager
Farah	NULL
Arun	Farah
Bhavana	Farah
Indira	Farah
Gita	Indira
Meera	Farah
Jai	Meera
Ravi	Farah
Sneha	Meera

Explanation

Think of two copies of the table: e (the employee) and m (the manager). Matching e.manager_id to m.emp_id fetches the manager's name. Farah has manager_id NULL (top of the hierarchy), so LEFT JOIN keeps her with manager NULL.

Example 2: Pairs of employees in the same department

SQL Query

```
SELECT a.emp_name AS emp1, b.emp_name AS emp2, a.dept_id
FROM employees a
JOIN employees b
  ON a.dept_id = b.dept_id AND a.emp_id < b.emp_id;
```

Output (5 rows)

emp1	emp2	dept_id
Farah	Arun	1
Farah	Bhavana	1
Arun	Bhavana	1
Indira	Gita	2
Meera	Jai	3

Explanation

Joining on the same dept_id pairs colleagues. The condition a.emp_id < b.emp_id stops an employee being paired with themselves and removes mirrored duplicates (Arun-Bhavana and Bhavana-Arun).

Topic 7: Multiple-Table JOIN

What it is: You can chain as many joins as needed. Each JOIN adds one more table, linked to a table already in the query. SQL builds the result step by step, left to right.

```
FROM   t1
JOIN   t2 ON t1.key = t2.key
JOIN   t3 ON t2.key = t3.key
JOIN   t4 ON t3.key = t4.key
```

Example 1: Order lines with customer and product (4 tables)

SQL Query

```
SELECT c.customer_name, o.order_id, p.product_name, oi.quantity
FROM customers c
JOIN orders o       ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id    = oi.order_id
JOIN products p     ON oi.product_id = p.product_id;
```

Output (9 rows)

customer_name	order_id	product_name	quantity
Ramesh	1	Laptop	1
Ramesh	1	Mouse	2
Ramesh	2	Phone	1
Suresh	3	Mouse	1
Suresh	3	Keyboard	1
Suresh	6	Phone	2
Lakshmi	4	Chair	2
Kiran	5	Laptop	1
Kiran	5	Keyboard	2

Explanation

customers links to orders, orders to order_items, and order_items to products. order_items is a 'bridge' table that connects orders to products (one order can contain many products).

Example 2: Mixing join types

SQL Query

```
SELECT c.customer_name, o.order_id, r.region_name
FROM customers c
JOIN regions r       ON c.region_id = r.region_id
LEFT JOIN orders o   ON c.customer_id = o.customer_id;
```

Output (8 rows)

customer_name	order_id	region_name
Ramesh	1	South
Ramesh	2	South
Suresh	3	North
Suresh	6	North
Lakshmi	4	South
Kiran	5	East
Priya	NULL	North
Vikram	NULL	West

Explanation

Customers are joined to regions with an INNER JOIN, then LEFT JOIN keeps customers who have no orders (Priya and Vikram show order_id NULL). Each join in the chain can be a different type.

Topic 8: JOIN with Aggregation

What it is: After joining tables you can apply aggregate functions (COUNT, SUM, AVG, MIN, MAX) to the combined rows. Without GROUP BY, the aggregate summarises the whole joined result into one row.

Example 1: Total value of all sales

SQL Query

```
SELECT SUM(oi.quantity * p.price) AS total_sales
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id;
```

Output (1 row)

total_sales
196,000

Explanation

Quantity lives in order_items and price lives in products, so a join is needed before multiplying. Each row gives quantity x price, and SUM adds them all.

Example 2: Number of orders and customers

SQL Query

```
SELECT COUNT(*) AS order_count, COUNT(DISTINCT customer_id) AS customers_who_ordered
FROM orders;
```

Output (1 row)

order_count	customers_who_ordered
6	4

Explanation

COUNT(DISTINCT ...) counts each customer once, even if they placed many orders. (This one uses a single table to show the aggregate idea clearly.)

Tip: COUNT(*) counts rows, including NULL-filled rows from a LEFT JOIN. COUNT(column) ignores NULLs, so use COUNT(o.order_id) to count real orders.

Topic 9: JOIN with GROUP BY

What it is: Join the tables first, then GROUP BY a column to get one summary row per group. Every non-aggregated column in SELECT must appear in GROUP BY.

Example 1: Employees per department (including empty ones)

SQL Query

```
SELECT d.dept_name, COUNT(e.emp_id) AS emp_count
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

Output (4 rows)

dept_name	emp_count
Finance	0
HR	2
IT	3
Sales	2

Explanation

LEFT JOIN keeps Finance. COUNT(e.emp_id) ignores the NULL, so Finance correctly shows 0. COUNT(*) would wrongly show 1 for Finance.

Example 2: Average salary per department

SQL Query

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

Output (3 rows)

dept_name	avg_salary
HR	45,000
IT	98,333.33
Sales	50,000

Explanation

Employees are matched to department names, then grouped by name so AVG is calculated for each department.

Common mistake

```
SELECT d.dept_name, COUNT(*)
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

COUNT(*) counts the single NULL-filled row for Finance as 1 employee.

Correct version

```
SELECT d.dept_name, COUNT(e.emp_id)
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

Topic 10: JOIN with HAVING

What it is: After JOIN and GROUP BY, HAVING filters the groups using aggregate values. Use WHERE for row conditions before grouping and HAVING for group conditions after grouping.

Execution order: FROM + JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

Example 1: Departments with more than 1 employee

SQL Query

```
SELECT d.dept_name, COUNT(e.emp_id) AS emp_count
FROM departments d
JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name
HAVING COUNT(e.emp_id) > 1;
```

Output (3 rows)

dept_name	emp_count
HR	2
IT	3
Sales	2

Explanation

IT has 3, HR has 2 and Sales has 2, so all three pass. Finance is not shown because the INNER JOIN already dropped it.

Example 2: Customers whose total purchases exceed 50,000

SQL Query

```
SELECT c.customer_name, SUM(oi.quantity * p.price) AS total_spent
FROM customers c
JOIN orders o       ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id    = oi.order_id
JOIN products p     ON oi.product_id = p.product_id
GROUP BY c.customer_name
HAVING SUM(oi.quantity * p.price) > 50000;
```

Output (2 rows)

customer_name	total_spent
Kiran	63,000
Ramesh	81,000

Explanation

Four tables are joined, spending is summed per customer, and HAVING keeps only customers above 50,000. Ramesh (81,000) and Kiran (63,000) qualify; Suresh (42,000) and Lakshmi (10,000) do not.

PART B: PRACTICE QUESTIONS

Q1. Display employees with department names

SQL Query

```
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

Output (7 rows)

emp_name	dept_name
Farah	IT
Arun	IT
Bhavana	IT
Indira	HR
Gita	HR
Meera	Sales
Jai	Sales

Explanation

INNER JOIN matches each employee to their department through dept_id. Only employees that have a department are shown (7 of 9).

Q2. Display students with course names

SQL Query

```
SELECT s.student_name, c.course_name
FROM students s
INNER JOIN courses c ON s.course_id = c.course_id;
```

Output (4 rows)

student_name	course_name
Anil	SQL
Bindu	SQL
Chitra	Python
Deepak	Python

Explanation

Matches students.course_id to courses.course_id. Esha (no course) is left out. To include her, use LEFT JOIN.

Q3. Find employees without departments

SQL Query

```
SELECT e.emp_id, e.emp_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

Output (2 rows)

emp_id	emp_name
8	Ravi
9	Sneha

Explanation

LEFT JOIN keeps all employees; those with no matching department get NULL in d.dept_id. WHERE d.dept_id IS NULL keeps just those. Alternative: WHERE e.dept_id IS NULL, which finds NULL values but would miss an employee whose dept_id points to a department that does not exist.

Q4. Find departments without employees

SQL Query

```
SELECT d.dept_id, d.dept_name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;
```

Output (1 row)

dept_id	dept_name
4	Finance

Explanation

Now departments is the left table. Departments with no matching employee have NULL in e.emp_id. Finance is the only one.

Q5. Display customers with their orders

SQL Query

```
SELECT c.customer_name, o.order_id, o.order_date
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;
```

Output (6 rows)

customer_name	order_id	order_date
Ramesh	1	2026-01-05
Ramesh	2	2026-02-10
Suresh	3	2026-02-12
Suresh	6	2026-03-20
Lakshmi	4	2026-03-01
Kiran	5	2026-03-15

Explanation

Each order row is matched to its customer. Customers with no orders do not appear. Ramesh and Suresh each have two orders, so they appear twice.

Q6. Display orders with product names

SQL Query

```
SELECT o.order_id, p.product_name, oi.quantity
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p     ON oi.product_id = p.product_id;
```

Output (9 rows)

order_id	product_name	quantity
1	Laptop	1
1	Mouse	2
2	Phone	1
3	Mouse	1
3	Keyboard	1
4	Chair	2
5	Laptop	1
5	Keyboard	2
6	Phone	2

Explanation

Orders and products are not directly linked. The bridge table order_items holds both order_id and product_id, so we join through it.

Q7. Find customers who never ordered

SQL Query

```
SELECT c.customer_id, c.customer_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Output (2 rows)

customer_id	customer_name
5	Priya
6	Vikram

Explanation

LEFT JOIN keeps all customers; those without orders have NULL order_id. Priya and Vikram never ordered.

Q8. Find products that were never sold

SQL Query

```
SELECT p.product_id, p.product_name
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.item_id IS NULL;
```

Output (1 row)

product_id	product_name
6	Desk

Explanation

Same anti-join pattern. Products with no matching row in order_items were never sold. Desk is the only one.

Q9. Join three tables (orders, customers, regions)

SQL Query

```
SELECT o.order_id, c.customer_name, r.region_name
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN regions r   ON c.region_id   = r.region_id;
```

Output (6 rows)

order_id	customer_name	region_name
1	Ramesh	South
2	Ramesh	South
3	Suresh	North
4	Lakshmi	South
5	Kiran	East
6	Suresh	North

Explanation

First orders is joined to customers, then that result is joined to regions. Each ON links the new table to one already present. Result: every order with the customer's name and region.

Q10. Join four tables

SQL Query

```
SELECT c.customer_name, o.order_id, p.product_name,
       oi.quantity, oi.quantity * p.price AS line_total
FROM customers c
JOIN orders o       ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id    = oi.order_id
JOIN products p     ON oi.product_id = p.product_id;
```

Output (9 rows)

customer_name	order_id	product_name	quantity	line_total
Ramesh	1	Laptop	1	60,000
Ramesh	1	Mouse	2	1,000
Ramesh	2	Phone	1	20,000
Suresh	3	Mouse	1	500
Suresh	3	Keyboard	1	1,500
Suresh	6	Phone	2	40,000
Lakshmi	4	Chair	2	10,000
Kiran	5	Laptop	1	60,000
Kiran	5	Keyboard	2	3,000

Explanation

Chains customers, orders, order_items and products. Each row is one purchased line, with its total calculated as quantity x price.

Q11. Find each employee's manager using SELF JOIN

SQL Query

```
SELECT e.emp_name AS employee, m.emp_name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

Output (9 rows)

employee	manager
Farah	NULL
Arun	Farah
Bhavana	Farah
Indira	Farah
Gita	Indira
Meera	Farah
Jai	Meera
Ravi	Farah
Sneha	Meera

Explanation

The employees table is used twice: e for the employee and m for the manager. e.manager_id equals the manager's emp_id. LEFT JOIN keeps Farah, who has no manager (NULL).

Q12. Calculate total sales by customer

SQL Query

```
SELECT c.customer_name,
       SUM(oi.quantity * p.price) AS total_sales
FROM customers c
JOIN orders o       ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id    = oi.order_id
JOIN products p     ON oi.product_id = p.product_id
GROUP BY c.customer_name
ORDER BY total_sales DESC;
```

Output (4 rows)

customer_name	total_sales
Ramesh	81,000
Kiran	63,000
Suresh	42,000
Lakshmi	10,000

Explanation

Join to get quantity and price for each line, multiply them, then GROUP BY customer and SUM. To also list customers with zero sales, change the joins after customers to LEFT JOIN and wrap the SUM in COALESCE(..., 0).

Q13. Calculate total sales by product

SQL Query

```
SELECT p.product_name,
       SUM(oi.quantity)           AS units_sold,
       SUM(oi.quantity * p.price) AS total_sales
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_name
ORDER BY total_sales DESC;
```

Output (5 rows)

product_name	units_sold	total_sales
Laptop	2	120,000
Phone	3	60,000
Chair	2	10,000
Keyboard	3	4,500
Mouse	3	1,500

Explanation

Grouping by product gives units and revenue for each. Desk does not appear because it has no sales rows. Use LEFT JOIN to show it with NULL or 0.

Q14. Calculate total sales by region

SQL Query

```
SELECT r.region_name,
       COALESCE(SUM(oi.quantity * p.price), 0) AS total_sales
FROM regions r
LEFT JOIN customers c   ON r.region_id   = c.region_id
LEFT JOIN orders o      ON c.customer_id = o.customer_id
LEFT JOIN order_items oi ON o.order_id   = oi.order_id
LEFT JOIN products p    ON oi.product_id = p.product_id
GROUP BY r.region_name
ORDER BY total_sales DESC;
```

Output (4 rows)

region_name	total_sales
South	91,000
East	63,000
North	42,000
West	0

Explanation

The path is regions → customers → orders → order_items → products. LEFT JOINs keep West, which has no sales, and COALESCE turns its NULL sum into 0. South = 81,000 + 10,000 = 91,000.

Q15. Compare INNER JOIN and LEFT JOIN

A) INNER JOIN

```
SELECT c.customer_name, o.order_id
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;
```

Output (6 rows)

customer_name	order_id
Ramesh	1
Ramesh	2
Suresh	3
Suresh	6
Lakshmi	4
Kiran	5

B) LEFT JOIN

```
SELECT c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

Output (8 rows)

customer_name	order_id
Ramesh	1
Ramesh	2
Suresh	3
Suresh	6
Lakshmi	4
Kiran	5
Priya	NULL
Vikram	NULL

Comparison

Point	INNER JOIN	LEFT JOIN
Rows returned	Only matching rows from both tables	All left-table rows plus matches from the right
Unmatched left rows	Dropped	Kept, right columns are NULL
Unmatched right rows	Dropped	Dropped
Result here	6 rows (Priya and Vikram missing)	8 rows (Priya and Vikram shown with NULL order)
Typical use	Data that must exist in both tables	Find missing or unmatched records, or keep all of one table
Row count	Never more than a LEFT JOIN on the same condition	At least as many rows as the left table

Conclusion: The only difference is what happens to left-table rows with no match. INNER JOIN discards them, LEFT JOIN keeps them with NULLs. Choose INNER JOIN when you only want complete matches, and LEFT JOIN when you must not lose any rows from the first table (or want to find the ones that have no match).

Quick Revision

I want to...	Use

Only rows that match in both tables	INNER JOIN
All rows from the first table	LEFT JOIN
Find rows with no match	LEFT JOIN ... WHERE right_key IS NULL
All combinations of two tables	CROSS JOIN
Compare rows within one table	SELF JOIN (two aliases)
Count only real matches	COUNT(right_table.column), not COUNT(*)
Filter groups after joining	GROUP BY ... HAVING